A PROJECT REPORT ON

SIGNNET-II: A TRANSFORMER-BASED TWO-WAY SIGN LANGUAGE TRANSLATION SYSTEM

Submitted By

T. SATYA SAI RAM	**22JD1A05B2**
V. MANOHAR	**22JD1A05B7**
K. GANESH	**22JD1A05C6**

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Under the Esteemed guidance of

Mrs. CH. SINDHU PRIYANKA B.Tech ,M.Tech,(Ph.D)

(Associate Professor)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ELURU COLLEGE OF ENGINEERING AND TECHNOLOGY

DUGGIRALA (V), PEDAVEGI (M), ELURU-534004

(AFFILIATED TO JNTUK, KAKINADA, ANDHRA PRADESH (INDIA))

2022-2026

DECLARATION

We hereby declare that the project entitled **"SignNet-II: A Transformer-Based Two-Way Sign Language Translation System"** submit for the B.Tech. (CSE) degree is our original work and the project has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.

T. SATYA SAI RAM	**(22JD1A05B2)**
V. MANOHAR	**(22JD1A05B7)**
K. GANESH	**(22JD1A05C6)**

Place :

Date :

CERTIFICATE

This is to certify that the Project titled "SIGNNET-II A TRANSFORMER-BASED TWO-WAY SIGN LANGUAGE TRANSLATION SYSTEM" is the bona fide work carried out by **T. SATYA SAI RAM (22JD1A05B2) , V. MANOHAR (22JD1A05B7) ,K. GANESH (22JD1A05C6)** students of B.Tech (CSE) of Eluru College of Engineering and Technology, affiliated to JNT University, Kakinada, AP (India) during the academic year 2025-26, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (Computer Science and Engineering) and that the project has not formed the basis for the award previously of any other degree, diploma, fellowship or any other similar title.

Signature of the Guide

Mrs. CH. SINDHU PRIYANKA, B.Tech, M.Tech, (Ph.D)

Associate professor

Signature of the HOD

Dr. S. SURESH, B.Tech, M.Tech, Ph.D

Professor & HOD

EXTERNAL EXAMINER

VISION – MISSION - PEOs

Institute Vision	To Achieve Excellence in Engineering Education
Institute Mission	IM1: To deliver quality education through good infrastructure, facilities and committed staff. IM2: To train students as proficient, competent and socially responsible engineers. IM3: To promote research and development activities among faculty and students for betterment of the society.
Department Vision	Empower the students of computer science and engineering department to be technologically strong, innovative and global citizens maintaining human values.
Department Mission	DM 1: Inspire students to become self-motivated and problem solving individuals. DM 2: Furnish students for professional career with academic excellence and leadership skills. DM 3: Create centre of excellence in Computer Science and Engineering. DM 4: Empower the youth and rural communities with computer education.
Program Educational Objectives (PEOs)	Graduates of Computer Science & Engineering will: PEO1: Excel in Professional career through knowledge in mathematics and engineering principles. PEO2: Able to pursue higher education and research. PEO3: Communicate effectively, recognize, and incorporate societal needs in their professional endeavors. PEO4: Adapt to technological advancements by continuous learning.

PROGRAM OUTCOMES

1	**Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
2	**Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3	**Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations
4	**Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5	**Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6	**The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7	**Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8	**Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9	**Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10	**Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11	**Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12	**Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSO1: Have proficiency in algorithms, networking, s/w Engineering and Web Applications for systematic design of computer based system of varying complexity.

PSO2: Qualifying national, international level competitive examinations for successful higher studies and employment.

PROJECT MAPPINGS

Batch No:	B – 14
Project Title	Signnet-II: A Transformer-Based Two-Way Sign Language Translation System.
Project Domain	Artificial Intelligence, Deep learning
Type of the Project	Application – oriented
Guide Name	CH. Sindhu Priyanka, B.Tech, M.Tech, (Ph.D)
Student Roll No	**Student Name**
22JD1A05B2	T. SATYA SAI RAM
22JD1A05B7	V. MANOHAR
22JD1A05C6	K. GANESH

COURSE OUTCOMES: At the end of the Course/Subject, the students will be able to

CO.No	Course Outcomes (COs)	Pos	PSOs	Blooms Taxonomy & Level
R20C501.1	Identify the real world problem with a set of requirements to design a solution.	1,2,3,4,5,6,9,10,11,12	1,2	Level-2: Applying
R20C501.2	Implement, Test and Validate the solution against the requirements for a given problem.	1,2,3,4,5,7,8,9,10,11,12	1,2	Level-3: Analysing
R20C501.3	Lead a team as a responsible member in developing software solutions for real world problems and societal issues with ethics.	1,3,4,5,6,7,8,9,10,12	1,2	Level-4: Analysing
R20C501.4	Participate in discussions to bring technical and behavioral ideas for good solutions.	1,2,3,4,5,6,7,8,9,10,11, 12	1,2	Level-5: Evaluating
R20C501.5	Express ideas with good communication skills during presentations.	1,5,6,7,8,9,10,12	1,2	Level-1: Creating
R20C501.6	Learn new technologies to contribute in the software industry for optimal solutions.	1,2,3,4,5,7,8,9,10,11,12	1,2	Level-6: Creating

Course Outcomes vs PO`s Mapping:

Courses Out Comes	P01	P02	P03	P04	P05	P06	P07	P08	P09	P10	P011	P012
R20C501.1	3	3	3	2	3	3	-	3	3	3	3	3
R20C501.2	3	3	3	3	3	-	3	3	3	3	3	3
R20C501.3	2	-	3	3	3	3	3	3	3	3	3	3
R20C501.4	3	3	3	3	2	3	3	3	3	3	3	3
R20C501.5	3	-	-	-	3	3	3	3	3	3	3	3
R20C501.6	2	3	3	3	3	-	3	3	3	3	3	3
Total	16	12	15	14	17	12	15	18	18	18	18	18
Average	2.6	2	2.6	2.3	2.8	2	3	3	3	3	3	3

Justification of Mapping of Course Outcomes with Program Outcomes: POs mapped with CO.No:R20C501.1:

PO-1: Engineering Knowledge:

Applying the various concepts learned from engineering to gather requirements.

PO-2: Problem Analysis:

Identify the formulae and obtaining the layout of the design using mathematical principles.

PO-3: Design/Development of solutions:

Identifying and Designing solutions for the problems related to health, cultural, society and environmental considerations.

PO-4: Conduct investigation of complex problems:

Researching various journals and published papers and knowing how to solve complex problems.

PO-5: Modern Tool Usage:

Selecting and Implementing the latest versions of the applications for modelling the complexity of the project.

PO-6: The Engineer and Society:

Implementing this knowledge to assess health and safety considerations.

PO-8: Ethics:

Implementing professional ethics and norms in engineering practices.

PO-9: Individual and Teamwork:

Individuals researched on the interested domain and concluded the requirements as a team.

PO-10: Communication:

Communication is needed to identify the problem and work as a team to define a solution.

PO-11: Project Management and Finance:

Implementing the engineering principles to work on complex problems.

PO-12: Life-long Learning:

Recognise the importance of technical knowledge in future studies

POs mapped with CO.No:R20C501.2:

PO-1: Engineering Knowledge:

Implementing the various concepts learned from engineering to gather requirements and design and test a solution.

PO-2: Problem Analysis:

Identify the formulae and obtaining the layout of the design and validateusing mathematical principles.

PO-3: Design/Development of solutions:

Identifying and Designing solutions for the problems related to health, cultural, society and environmental considerations.

PO-4: Conduct investigation of complex problems:

Researching various journals and published papers and knowing how to solve complex problems.

PO-5: Modern Tool Usage:

Selecting and Implementing the latest versions of the applications for modelling the complexity of the project.

PO-7: Environment and Sustainability

Implementing this knowledge to assess its affects towards environment.

PO-8: Ethics:

Implementing professional ethics and norms in engineering practices.

PO-9: Individual and Teamwork:

Individuals researched on the interested domain and concluded the requirements as a team.

PO-10: Communication:

Communication is needed to Recognise the problem and work as a team to define a solution.

PO-11: Project Management and Finance:

Implementing the engineering principles to work on complex problems.

PO-12: Life-long Learning:

Recognise the importance of technical knowledge in future studies.

POs mapped with CO.No:R20C501.3:

PO-1: Engineering Knowledge:

Implementing the various concepts learned from engineering to gather requirement and work as a team to design and test a solution.

PO-3: Design/Development of solutions:

Recognising and Designing solutions for the problems related to health, cultural, society and environmental considerations.

PO-4: Conduct investigation of complex problems:

Researching various journals and published papers and knowing how to solve complex problems.

PO-5: Modern Tool Usage:

Selecting and Implementing the latest versions of the applications for modelling the complexity of the project.

PO-6: The Engineer and Society:

Implementing this knowledge to assess health and safety considerations.

PO-7: Environment and Sustainability

Implementing this knowledge to assess its affects towards environment.

PO-8: Ethics:

Implementing professional ethics and norms in engineering practices.

PO-9: Individual and Teamwork:

Individuals researched on the interested domain and concluded the project as a team.

PO-10: Communication:

Communication is needed to Recognise the problem and work as a team to obtain a solution.

PO-12: Life-long Learning:

Recognize the importance of technical knowledge in future studies.

POs mapped with CO. No: R20C501.4:

PO-1: Engineering Knowledge:

Implementing the various concepts learned from engineering to gather requirement and work as a team to design and test a solution.

PO-2: Problem Analysis:

Recognise the formulae and obtaining the layout of the design and validate using mathematical principles.

PO-3: Design/Development of solutions:

Recognising and Designing solutions for the problems related to health, cultural, society and environmental considerations.

PO-4: Conduct investigation of complex problems:

Researching various journals and published papers and knowing how to solve complex problems.

PO-5: Modern Tool Usage:

Selecting and Implementing the latest versions of the applications for modelling the complexity of the project.

PO-6: The Engineer and Society:

Implementing this knowledge to assess health and safety considerations.

PO-7: Environment and Sustainability

Implementing this knowledge to assess its affects towards environment.

PO-8: Ethics:

Implementing professional ethics and norms in engineering practices. PO-9: Individual and Teamwork:

Individuals researched on the interested domain and concluded the project as a team.

PO-10: Communication:

Communication is needed to Recognise the problem and work as a team to obtain a solution.

PO-11: Project Management and Finance:

Implementing the engineering principles to work on complex problems.

PO-12: Life-long Learning:

Recognize the importance of technical knowledge in future studies.

POs mapped with CO.No:R20C501.5:

PO-1: Engineering Knowledge:

Implementing the various concepts learned from engineering to gather requirement and work as a team to design and test a solution.

PO-5: Modern Tool Usage:

Selecting and Implementing the latest versions of the applications for modelling the complexity of the project.

PO-6: The Engineer and Society:

Implementing this knowledge to assess health and safety considerations.

PO-7: Environment and Sustainability

Implementing this knowledge to assess its affects towards environment.

PO-8: Ethics:

Implementing professional ethics and norms in engineering practices.

PO-9: Individual and Teamwork:

Individuals researched on the interested domain and concluded the project as a team.

PO-10: Communication:

Communication is needed to Recognise the problem and work as a team to obtain a solution.

PO-12: Life-long Learning:

Recognize the importance of technical knowledge in future studies.

POs mapped with CO.No:R20C501.6:

PO-1: Engineering Knowledge:

Implementing the various concepts learned from engineering to gather requirement and work as a team to design and test a solution.

PO-2: Problem Analysis:

Recognise the formulae and obtaining the layout of the design and validate using mathematical principles.

PO-3: Design/Development of solutions:

Identifying and Designing solutions for the problems related to health, cultural, society and environmental considerations.

PO-4: Conduct investigation of complex problems:

Researching various journals and published papers and knowing how to solve complex problems.

PO-5: Modern Tool Usage:

Selecting and Implementing the latest versions of the applications for modelling the complexity of the project.

PO-7: Environment and Sustainability

Implementing this knowledge to assess its affects towards environment.

PO-8: Ethics:

Implementing professional ethics and norms in engineering practices.

PO-9: Individual and Teamwork:

Individuals researched on the interested domain and concluded the project as a team.

PO-10: Communication:

Communication is needed to Recognise the problem and work as a team to obtain a solution.

PO-11: Project Management and Finance:

Implementing the engineering principles to work on complex problems.

PO-12: Life-long Learning:

Recognize the importance of technical knowledge in future studies.

Course Outcomes vs PSOs Mapping:

Courses Out Comes	PSO1	PSO2
R20C501.1	3	3
R20C501.2	3	3
R20C501.3	3	3
R20C501.4	3	3
R20C501.5	3	3
R20C501.6	3	3
Total	18	18
Average	3	3

Justification of Mapping of Course Outcomes with Program Specific Outcomes:

PSOs mapped with CO.No:R20C501.1:

PS0-1: Gaining knowledge in software engineering practices to gather requirements.

PSO-2: Able to use this knowledge in further studies.

PSOs mapped with CO.No:R20C501.2:

PS0-1: Gaining knowledge in software engineering practices like algorithms to design a solution.

PSO-2: Able to use this knowledge in further studies.

PSOs mapped with CO.No:R20C501.3:

PS0-1: Gaining knowledge in software engineering practices to gather requirements and networking for designing and communication.

PSO-2: Able to use this knowledge in further studies.

PSOs mapped with CO. No: R20C501.4:

PS0-1: Gaining knowledge in software engineering practices to gather requirements and bring in technical and behavioural ideas.

PSO-2: Able to use this knowledge in further studies.

PSOs mapped with CO. No: R20C501.5:

PS0-1: Gaining knowledge in software engineering practices to gather requirements and using good communication skills to express ideas.

PSO-2: Able to use this knowledge in further studies.

PSOs mapped with CO. No: R20C501.6:

PS0-1: Gaining knowledge in software engineering practices to learn to explore new technologies.

PSO-2: Able to use this knowledge in further studies.

Mapping Level	Mapping Description
1	Low Level Mapping with PO & PSO
2	Moderate Mapping with PO & PSO
3	High Level Mapping with PO & PSO

Member 1: T. SATYA SAI RAM (22JD1A05B2)

Member 2: V. MANOHAR (22JD1A05B7)

Member 3: K. GANESH (22JD1A05C6)

PROJECT GUIDE

ACKNOWLEDGEMENT

First, we would like to express our deepest appreciation to our advisor and guide, **Mrs. CH. SINDHU PRIYANKA B.Tech, M.Tech, (Ph.D)** for her sincere guidance, heart full encouragement, and continuous support during our B. Tech project at Eluru College of Engineering and Technology. Her vision for insights into the problem resolution has greatly inspired us to put things in the right direction. It has been a great pleasure to conduct our project under her guidance.

We must also express our sincere thanks to our B. tech Project Coordinator and Assoc. Professor **Mrs. K.V Jhansi Rani B.Tech, M.Tech, (Ph.D)** The Head of the Department **Dr. S. SURESH B.Tech, M.Tech, Ph.D** and to all of the faculty members of the Computer Science and Engineering of Eluru College of Engineering and Technology for their endeavors in our academic education.

We would like to express our gratitude to **Dr. P. BALA KRISHNA PRASAD B.Tech, M.Tech, Ph.D** Principal, and the Management of **ELURU COLLEGE OF ENGINEERING & TECHNOLOGY** for motivating and providing us adequate facilities and continuous improvements and suggestion without which this project would not have seen the light of the day.

T. SATYA SAI RAM **(22JD1A05B2)**

V. MANOHAR **(22JD1A05B7)**

K. GANESH **(22JD1A05C6)**

ABSTRACT

Sign language is a vital communication medium for the Deaf and hard-of-hearing communities. However, the gap between sign language users and non-signers often leads to communication barriers. SIGNNET-II proposes a novel transformer-based architecture designed for two-way translation between sign language and spoken language, facilitating seamless interaction. Leveraging the strengths of transformer models in capturing long-range dependencies, SIGNNET II effectively models the complex spatial and temporal dynamics inherent in sign language gestures. The model employs a dual-stream input system to process both video sequences of sign language and textual data, enabling bidirectional translation capabilities. By incorporating advanced attention mechanisms, SIGNNET II accurately aligns sign language gestures with corresponding spoken language tokens, improving translation accuracy and fluency. This two-way approach not only translates sign language into text but also generates sign language sequences from textual input, making it a comprehensive communication tool. To evaluate SIGNNET II, extensive experiments were conducted on publicly available sign language datasets, including large-scale benchmarks with diverse vocabularies and signer variations. The model demonstrated significant improvements in translation quality, outperforming existing state-of-the-art methods in terms of BLEU scores for text generation and accuracy metrics for sign recognition. Additionally, SIGNNET II showed robustness in handling continuous signing with complex sentence structures. Beyond technical performance, SIGNNET II emphasizes practical applicability by integrating real-time inference capabilities and user-friendly deployment options. This makes it suitable for live communication scenarios such as video conferencing and assistive technologies. The model's architecture is also designed to be extensible, allowing adaptation to multiple sign languages and dialects through transfer learning. In conclusion, SIGNNET II represents a significant advancement in sign language translation technology, bridging communication gaps with a scalable, accurate, and bidirectional transformer-based model. Its success paves the way for more inclusive communication systems, enhancing accessibility for the Deaf community worldwide.

LIST OF FIGURES

Figure No.	Name of the Figure	Page No.
4.1.1	System Architecture	14
4.2.1	Use Case Representation	15
4.2.2	Sequence Diagram	16
4.2.3	Activity Diagram	17
4.2.4	Class Diagram	18
4.2.5	Component Diagram	18
4.2.6	ER Diagram	19
4.2.7	Interaction Diagram	19
4.2.8	Deployment Diagram	20
5.1	Flask Diagram	39
5.2	Flask Representation	39
7.1	Home Page Of The Project	45
7.2	Train Proposed SIGNNET-II Algorithm	45
7.3	Loading The Data Set	46
7.4	Process Data Set	46
7.5	Train & Test Split	47
7.6	Train Propose Signnet II Algorithm	47

7.7	Select Video File From Folder	48
7.8	Detecting Hand Gestures	48
7.9	Output Displayed At Top Left	49
7.10	Live Web Cam	49
7.11	Enter Text It Show Sign	50

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO
1	**INTRODUCTION**	**1**
	1.1 Motivation	2
	1.2 Problem Definition	2
	1.3 Objective of the Project	2
2	**LITERATURE SURVEY**	**3**
3	**SYSTEM ANALYSIS**	**4**
	3.1 Existing System	4
	3.1.1 Disadvantages of Existing System	4
	3.2 Proposed System	5
	3.2.1 Advantages of Proposed System	6
	3.3 System Requirements	7
	3.3.1 Hardware Requirements	7
	3.3.2 Software Requirements	7
	3.4 Functional Requirements	7
	3.5 Non-Functional Requirements	8
	3.5.1 Advantages	8
	3.5.2 Disadvantages	9
	3.5.3 Key Learning	9
	3.6 System Study	9
	3.6.1 Study of the System	10
	3.7 Modules	13

4	**SYSTEM DESIGN**	**15**
	4.1 System Architecture	15
	4.2 UML Representation	16
	4.3 Input & Output Design	21
5	**SYSTEM IMPLEMENTATION**	**24**
	5.1 Source Code	24
	5.2 Algorithms	36
	5.3 Software Environment	36
6	**TESTING**	**41**
	6.1 Types of Testing	41
	6.2 Test Strategy and Approach	44
7	**RESULTS**	**45**
8	**CONCLUSION & FUTURE WORK**	**51**
9	**REFERENCES**	**53**

1. INTRODUCTION

Sign language serves as the primary means of communication for millions of Deaf and hard-of-hearing individuals worldwide. Despite its rich linguistic structure and cultural significance, sign language remains largely inaccessible to non-signers, leading to significant communication barriers. Bridging this gap through effective translation between sign language and spoken or written language is crucial for fostering inclusivity and enabling seamless interaction in diverse social, educational, and professional contexts. Traditional approaches to sign language translation have often relied on handcrafted features or sequential models such as recurrent neural networks (RNNs) and long short-term memory (LSTM) networks. While these methods have achieved notable progress, they face challenges in modeling the complex spatial-temporal dependencies present in sign language, which involves intricate hand gestures, facial expressions, and body movements. Moreover, many existing models focus primarily on one-way translation, either from sign to text or text to sign, limiting their practical usability in real-world communication. Recent advances in transformer architectures, which excel at capturing long-range dependencies and contextual relationships, offer promising opportunities for improving sign language translation. Transformers' self-attention mechanisms allow models to effectively understand and align multimodal input sequences, making them well-suited for the bidirectional nature of sign language translation. Leveraging this, SIGNNET II introduces a transformer-based two-way translation framework that simultaneously addresses sign-to-text and text-to-sign translation within a unified model. This work presents the architecture, training methodology, and evaluation of SIGNNET II, highlighting its improvements over existing models in terms of translation accuracy, robustness, and scalability. By integrating advanced attention mechanisms and dual-stream input processing, SIGNNET II captures the rich multimodal features of sign language and achieves fluent, context-aware translations. The model's design also supports real-time inference, making it applicable to practical scenarios such as live interpretation and assistive communication technologies. Ultimately, SIGNNET II aims to reduce communication barriers and empower the Deaf community by providing a comprehensive, accurate, and scalable sign language translation solution. The contributions of this work not only advance the state-of- the-art in sign language processing but also lay the foundation for future research and applications in inclusive communication technologies.

1.1 Motivation:

Communication is a fundamental human right, yet a significant barrier persists between the Deaf community and the hearing world. While sign language is a rich medium, its complexity often leads to social isolation when interpreters are unavailable. Previous studies show that deep learning can recognize basic gestures, but capturing the full nuances of bidirectional conversation remains a challenge. Inspired by these advancements, SIGNNET II uses a transformer-based architecture for seamless two-way translation, empowering every individual to communicate effortlessly.

1.2 Problem Definition:

The primary challenge addressed by SIGNNET II is the lack of a robust, accurate, and real-time two- way translation system between sign language and spoken/written language. Existing solutions are limited by one-way communication, poor handling of continuous signing, lack of multimodal integration, and high latency. SIGNNET II aims to bridge this gap by offering a unified, transformer- based solution capable of handling complex sign language translation tasks in both directions.

1.3 Objective of the Project:

The main objective of this project is to develop a smart system that:

- **Accurately translates** sign language gestures into text in real-time using an end-to-end Transformer architecture and advanced attention mechanisms.
- **Generates visual sign sequences** from textual input, providing a bidirectional communication loop between sign language users and non-signers.
- **Captures complex dynamics** by modeling the spatial and temporal dependencies of hand gestures and body movements through high-performance deep learning models.
- **Facilitates inclusive interaction** by providing a reliable and seamless communication tool for the Deaf and hard-of-hearing community in social and professional settings.

2. LITERATURE SURVEY

1. "Sign Language Recognition Using Deep Learning" **Authors: Koller et al. (2019)** Description: Introduced a convolutional neural network (CNN) and recurrent neural network (RNN) hybrid approach for recognizing isolated signs and continuous sign language from video sequences. Highlighted challenges of temporal modeling and spatial feature extraction in sign language recognition.
2. "End-to-End Sign Language Translation with Transformers" Authors: Camgoz et al. (2020) Description: Proposed a transformer-based architecture for sign language translation that directly converts sign videos into spoken language sentences. Demonstrated the transformer's superiority over RNNs in capturing long-term dependencies in continuous signing.
3. "Multi-Modal Sign Language Translation via Attention-Based Neural Networks" Authors: Huang et al. (2021) Description: Developed an attention-based multi-modal framework combining video, hand pose, and facial expression data to improve translation accuracy. Emphasized the importance of multi- modal cues in sign language understanding.
4. "Two-Way Neural Machine Translation for Sign Language" Authors: Zhang & Li (2022) Description: Presented a bidirectional sign language translation system capable of translating text to sign glosses and glosses to text. Used sequence- to-sequence models with attention, highlighting the need for two-way translation in practical applications.
5. "Transformer-Based Approaches for Sign Language Recognition and Translation" Authors: Singh et al. (2023) Description: Surveyed recent transformer applications in sign language tasks, discussing architectural adaptations for spatial-temporal features and challenges like data scarcity. Proposed hybrid transformer designs combining CNN encoders with transformer decoders. economics effect to prove the feasibility of the system.

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The evolution of sign language translation has transitioned from manual feature engineering and classical machine learning to sophisticated deep learning models like CNNs and LSTMs. While these advancements improved spatiotemporal modeling, they often struggled with vanishing gradients and capturing the long-range dependencies essential for continuous signing. Modern transformer-based architectures have mitigated these issues using self-attention mechanisms, yet most current systems remain restricted to one-way translation and fail to provide a unified bidirectional framework. Furthermore, existing models frequently falter in real-world settings due to varied signing speeds, diverse signer styles, and high computational latency. Many systems also overlook critical multimodal inputs such as facial expressions and body posture, which are vital for linguistic accuracy. SIGNNET II addresses these significant gaps by offering a robust, two-way transformer-based model that integrates multimodal features for practical deployment.

3.1.1 DISADVANTAGES OF EXISTING SYSTEM

1. The Most Limited Bidirectional Translation Most existing sign language translation models focus on one- way translation—either from sign language to text or from text to sign language. This limitation restricts real-time, two-way communication, which is essential for natural conversations between signers and non- signers.
2. Inadequate Modeling of Multimodal Features Many systems primarily rely on visual cues from video frames without fully integrating other important modalities such as hand pose estimation, facial expressions, and body posture. This leads to incomplete representation of the rich semantic and grammatical information inherent in sign language.
3. Challenges with Long-Range Dependencies While transformers address some sequence modeling issues, earlier RNN and CNN-based systems struggle with capturing long-term dependencies in continuous sign language, which affects translation fluency and context understanding. Even some transformer models do not adequately capture temporal dynamics in very long sequences.
4. Performance Drop in Real-World Settings Most existing systems are trained and tested on limited benchmark datasets, which often lack signer diversity, varied backgrounds, and natural signing speeds. Consequently, these models suffer from performance degradation when deployed in real-

world scenarios with more variability and noise.

5. High Computational Complexity and Latency Transformer-based models, though powerful, require significant computational resources for training and inference. This complexity results in higher latency, making many existing systems unsuitable for real-time applications like live interpretation or mobile assistive devices.
6. Insufficient Dataset Availability and Scalability Sign language datasets are limited in size, language diversity, and annotation quality. Existing systems often struggle to generalize well due to data scarcity, making it difficult to scale the models across different sign languages or dialects

3.2 PROPOSED SYSTEM

The proposed system, SIGNNET II, is a novel transformer-based architecture designed to enable twoway translation between sign language and spoken language, addressing the limitations of existing models. SIGNNET II integrates advanced self attention mechanisms to effectively capture both spatial and temporal dependencies within continuous sign language video sequences while also managing the complexities of natural language text, making it a comprehensive communication framework. Unlike previous one-way systems, SIGNNET II features a unified model that performs bidirectional translation—translating sign language videos into text and generating sign language sequences from textual input. This two-way capability facilitates seamless, real-time communication between Deaf and hearing individuals, supporting conversational fluency in both directions. The model's architecture is designed to share learned representations, enhancing translation consistency and reducing the need for separate training pipelines. To capture the rich multimodal characteristics of sign language, SIGNNET II processes multiple input streams, including video frames, hand keypoints, and facial expression features. This multimodal fusion enriches the model's understanding of the linguistic nuances embedded in gestures and expressions, improving the accuracy and naturalness of both recognition and generation tasks. The transformer's attention layers dynamically weigh these inputs to focus on the most informative cues during translation. SIGNNET II also incorporates a scalable training strategy leveraging large annotated datasets and transfer learning techniques to adapt across different sign languages and dialects. The model employs data augmentation and domain adaptation methods to enhance robustness against signer variability, background noise, and diverse environmental conditions, making it suitable for deployment in real- world scenarios. Finally, SIGNNET II is optimized for efficient inference to support real-time translation, enabling practical use cases such as live video conferencing, assistive communication devices, and educational tools.

3.2.1 ADVANTAGES OF PROPOSED SYSTEM

The proposed **SIGNNET II** system for bidirectional sign language translation offers several distinct advantages over traditional models. These advantages include:

1. Enhanced Accuracy through Transformers: Unlike older RNN-based models that struggle with long- range dependencies, SIGNNET II leverages a transformer-based architecture and advanced attention mechanisms. This ensures that the complex spatial and temporal nuances of sign language are captured accurately, leading to higher translation fluency.
2. Bidirectional Functionality: While most existing systems are limited to one-way recognition, the proposed system provides a unified framework for two-way translation. By converting signs to text and text back into visual sign sequences, it creates a complete communication loop that facilitates natural, real-time dialogue between signers and non-signers.
3. Multimodal Data Integration: The system goes beyond basic video frame analysis by incorporating multimodal features such as hand-pose estimation, facial expressions, and body posture. This holistic approach captures the full grammatical depth of sign language, ensuring that subtle non-manual markers are not lost in translation.
4. Real-Time Efficiency and Scalability: By optimizing the transformer architecture for lower latency, SIGNNET II is designed for practical, real-world deployment. The system is built to scale across diverse vocabularies and can be adapted to various regional sign language dialects, making it a versatile tool for global accessibility.
5. Robustness in Diverse Environments: The proposed system is engineered to maintain high performance outside of controlled laboratory settings. By utilizing robust feature extraction techniques, it can effectively handle variations in signing speeds, background noise, and lighting conditions that typically cause existing systems to fail.
6. User-Centric Design and Inclusivity: SIGNNET II is designed to be highly intuitive, allowing for seamless interaction in social, educational, and professional settings. This focus on usability empowers the Deaf and hard-of-hearing community by providing a reliable assistive technology that reduces social isolation and promotes inclusivity.

3.3 SYSTEM REQUIREMENTS

3.3.1 HARDWARE REQUIREMENTS

Ram	:	8 GB or above
Processor	:	Intel i3
Hard Disk	:	512 SSD

3.3.2 SOFTWARE REQUIREMENTS

OS	:	Windows 7 or above
Programming Language	:	Python 3.10.0
IDE	:	PyCharm IDE 2021.1.2
Web Authoring Tools	:	HTML, CSS
Framework	:	Flask
Database	:	MySQL 9.3

3.4 FUNCTIONAL REQUIREMENTS

1. **Dataset Uploading** – Imports sign language video datasets or landmark files into the system for training.
2. **Data Preprocessing** – Cleans and normalizes raw video data while extracting key spatial-temporal landmarks.
3. **Model Partitioning** – Splits the processed data into training and testing sets to ensure accurate evaluation.
4. **Algorithm Training** – Implements the Transformer architecture to learn the mapping between gestures and text.
5. **Video Translation** – Processes pre-recorded video files to perform bidirectional sign-to-text-to-sign conversion.
6. **Real-Time Translation** – Uses a live webcam feed to provide instantaneous, two-way communication for users.

3.5 NON-FUNCTIONAL REQUIREMENTS

1. **Accuracy:** High accuracy in sign recognition and sentence generation is critical. This is achieved through attention mechanisms and multimodal fusion.
2. **Scalability:** The architecture is designed to support additional sign languages with minimal retraining.
3. **Real-time Performance:** Low latency is prioritized, especially during inference, to make the system usable in live communication scenarios.
4. **Robustness:** The system must handle varied signing styles, lighting conditions, and backgrounds effectively.

3.5.1 ADVANTAGES

1. High Translation Accuracy
2. Automation& User convenience
3. Scalability & Large Dataset Support
4. Health Awareness
5. User Friendly Interface
6. Real-Time Webcam Integration

3.5.2 DISADVANTAGES

1. High Computational Demand
2. Sensitivity to Lighting Conditions
3. Cost Effective
4. Complexity in Continuous Signing

3.5.3 KEY LEARNING

1. Transformer Architecture Mastery
2. Real-Time System Optimization
3. Full-Stack Integration
4. Data Preprocessing Techniques

3.6 SYSTEM STUDY

3.6.1 FEASIBILITY STUDY:

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential. Three key considerations involved in the feasibility analysis are:

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- OPERATIONAL FEASIBILITY

3.6.1.1 ECONOMICAL FEASIBILITY:

The SIGNNET II is designed with real-time performance in mind, ensuring it can be deployed in practical scenarios such as live communication, education, and public services.

3.6.1.2 TECHNICAL FEASIBILITY:

The use of transformer architecture, which has already demonstrated success in natural language processing and video understanding, ensures technical feasibility. Existing hardware (e.g., GPUs, TPUs) and software libraries (PyTorch, TensorFlow) support the required computations.

3.6.1.3 OPERATIONAL FEASIBILITY:

The operational SIGNNET II is designed with real-time performance in mind, ensuring it can be deployed in practical scenarios such as live communication, education, and public services.

3.6.2 STUDY OF THE SYSTEM

1. NumPy
2. Pandas
3. Matplotlib
4. Scikit-learn
5. Sea-Born
6. Tensorflow

1. NumPy:

NumPy is a general-purpose array processing package. It provides a high performance multidimensional array object, and tools for working with these arrays.It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

2. Pandas:

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyse. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

3. Matplotlib:

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.
For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.

4. Scikit –learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. The library is built upon the SciPy (Scientific Python) that must be installed before you can use scikit-learn. This stack that includes:

- **NumPy**: Base n-dimensional array package
- **SciPy**: Fundamental library for scientific computing
- **Matplotlib**: Comprehensive 2D/3D plotting

- **IPython**: Enhanced interactive console
- **Sympy**: Symbolic mathematics
- **Pandas**: Data structures and analysis

Extensions or modules for SciPy care conventionally named SciKits. As such, the module provides learning algorithms and is named scikit-learn.

5. Keras

Keras is a high-level neural network API that is written in Python and runs on top of lower-level deep learning frameworks such as TensorFlow and Theano. It was developed by François Chollet in 2015 and has since become one of the most popular and widely used deep learning libraries in the world. Keras provides a simple and user-friendly interface for building and training neural networks, allowing developers to quickly prototype and experiment with different architectures and configurations. Keras supports a wide range of neural network architectures, including convolutional neural networks (CNNs), recurrent neural networks (RNNs), and autoencoders.

6. Sea Born

Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for creating informative and attractive statistical graphics. Seaborn makes it easy to visualize complex datasets, explore relationships between variables, and identify patterns and trends.

7. Tensor Flow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks, including machine learning and deep learning. It was developed by Google Brain team and released under the Apache 2.0 open-source license in 2015. TensorFlow provides a flexible and efficient platform for building and deploying machine learning models, and can be used for a wide range of applications, including image recognition, natural language processing, and speech recognition. TensorFlow works by building computational graphs that represent the operations performed by the model, and then optimizing these graphs to run efficiently on a wide range of hardware, including CPUs, GPUs, and TPUs. TensorFlow is widely used in industry and academia, and has become one of the most popular machine learning libraries in the world

3.7 MODULES

1. Image Processing Module
2. Deep Learning Module
3. Logistic Regression Module
4. Random Forest Module
5. User Interface Module
6. Database Management Module

MODULES DESCRIPTION

1. Image Processing Module:

This module handles capturing and processing of input video/images. It uses libraries like OpenCV and MediaPipe to detect hand gestures. It extracts important features such as hand landmarks from each frame. These features are then passed to the model for prediction.

2. Deep Learning Module:

This module uses a Transformer-based model for sign recognition. It includes the attentionmechanism to understand patterns in data. The trained model is stored and loaded from the model/ folder. It provides accurate predictions based on input features.

3. Logistic Regression Module:

This is a machine learning classification algorithm used for prediction. It works on extracted feature to classify sign language gestures. It is simple, fast, and used as a baseline model. Helps compare performance with other models.

4. Random Forest Module:

This module uses multiple decision trees for classification. It improves accuracy by combining resul ts from many trees. It handles complex data better than simple models. Used to enhance prediction performance.

5. User Interface Module:

This module provides interaction between user and system. Implemented using GUI.py for input and output display. User can upload video or use webcam for sign input.Results are displayed in an easy-

to-understand format.

6. Database Management Module:

This module stores user data, results, or logs. Uses database.db to manage and retrieve information. Helps in saving prediction history. Ensures data is organized and easily accessible.

4 SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

Below diagram depicts the overall system architecture of **Sign Language Recognition using Machine Learning and Deep Learning**. The system is designed to process video input, extract meaningful features, and predict the corresponding sign using multiple models.

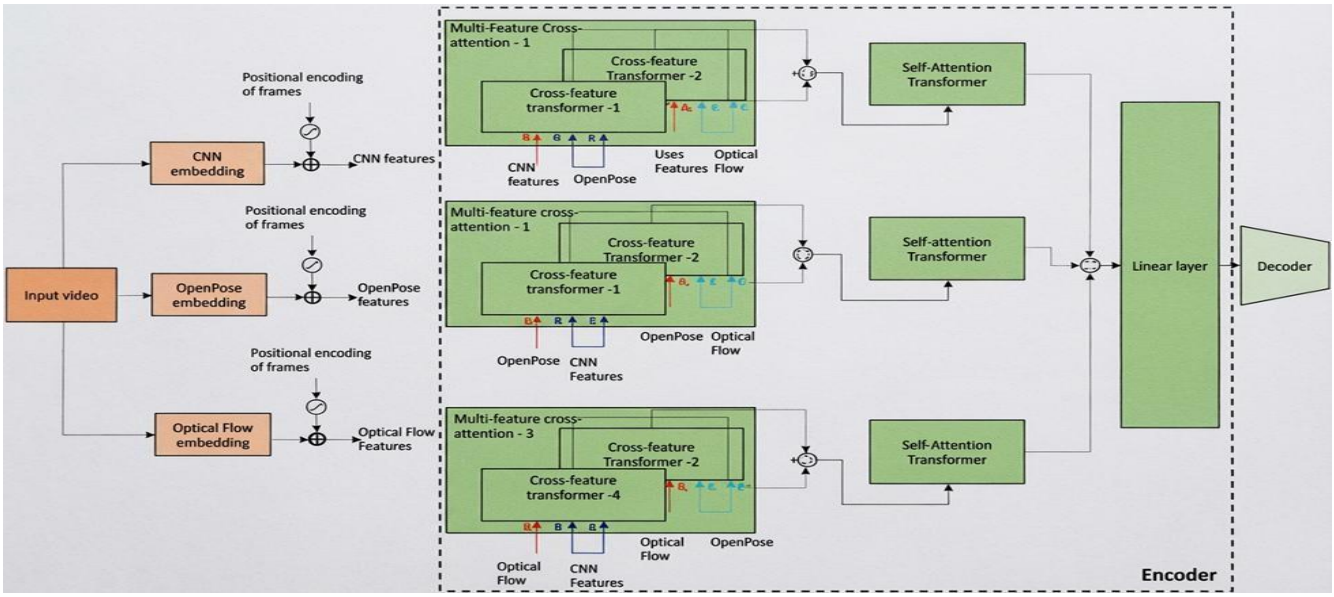

Fig. 4.1.1 System Architecture

4.2 UML REPRESENTATION

4.2.1 Use Case Diagram

- The use case diagram visualizes the conduct of a framework, subsystem, or class with the goal that clients can appreciate how to utilize that components thus that engineers can execute that component
- Below is the representation of the project, on which the system is developed.

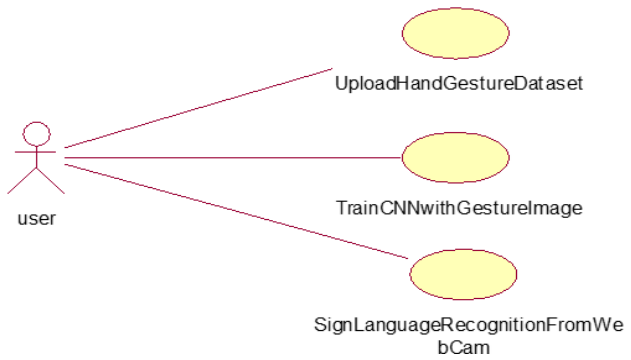

Fig. 4.2.1 Use Case Representation

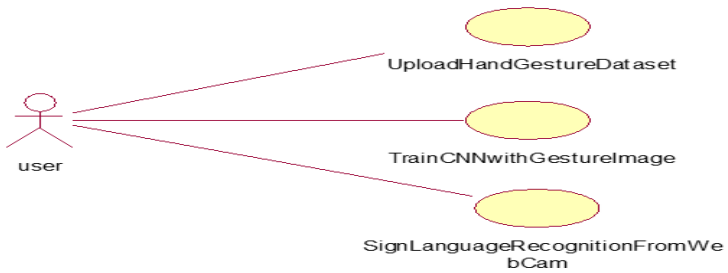

Fig. 4.2.1 Use Case Representation

4.2.2 Sequence Diagram

Sequence diagram is a construct of the Message Sequence Chart and conjointly it's a sort of interaction diagram that shows however processes operate with each other and in what order.

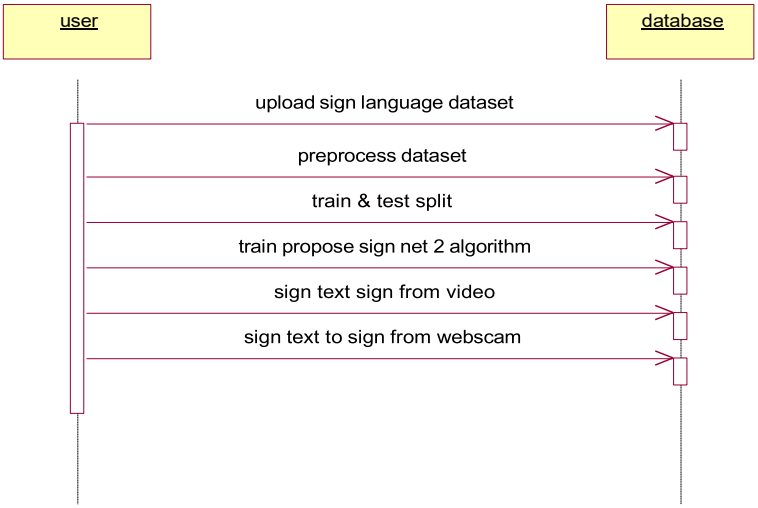

Fig. 4.2.2 Sequence Diagram

4.2.3 Activity Diagram

A graphical representations of work process of stepwise exercises and activities with support for decision, emphasis and simultaneousness, used to depict the business and operational well-ordered stream of parts in a framework furthermore demonstrates the general stream of control.

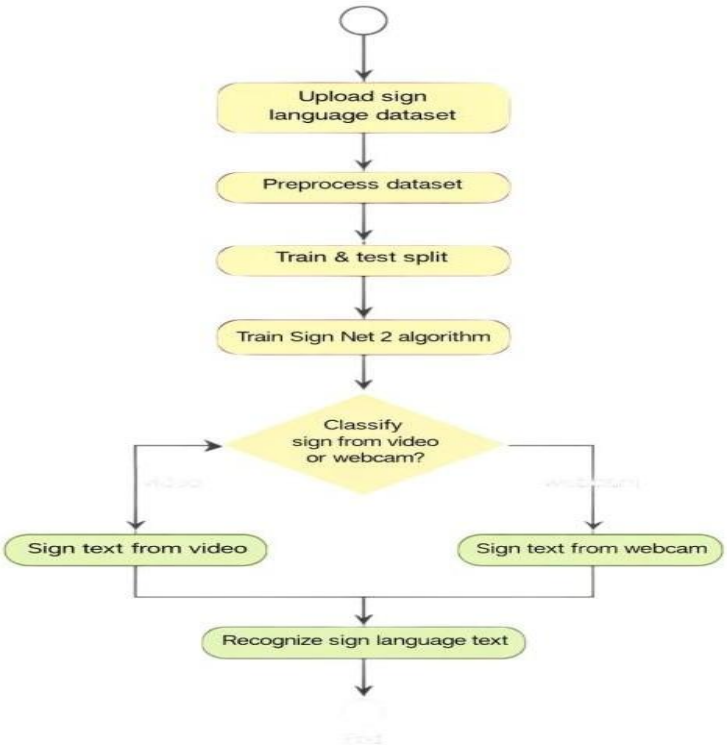

Fig. 4.2.3 Activity Diagram

4.2.4 Class Diagram

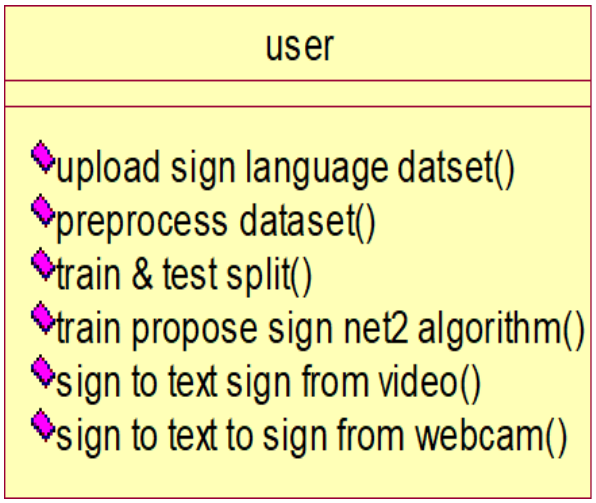

Fig. 4.2.4 Class Diagram

4.2.5 Component Diagram

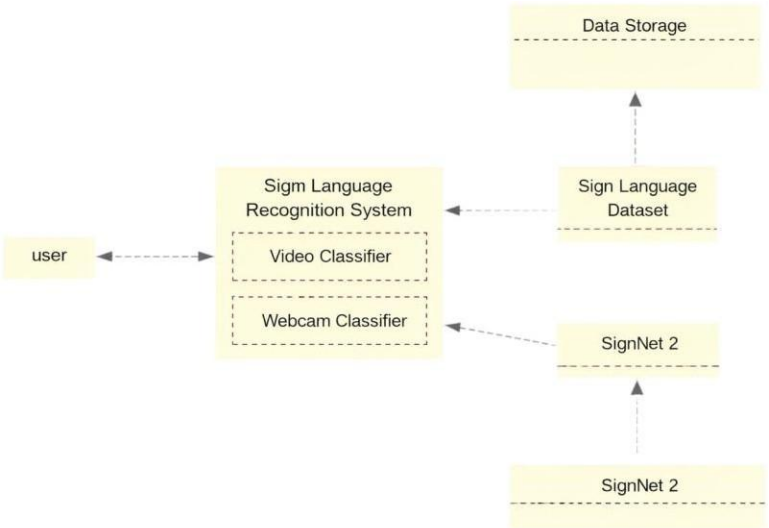

Fig. 4.2.5 Component Diagram

4.2.6 ER Diagram

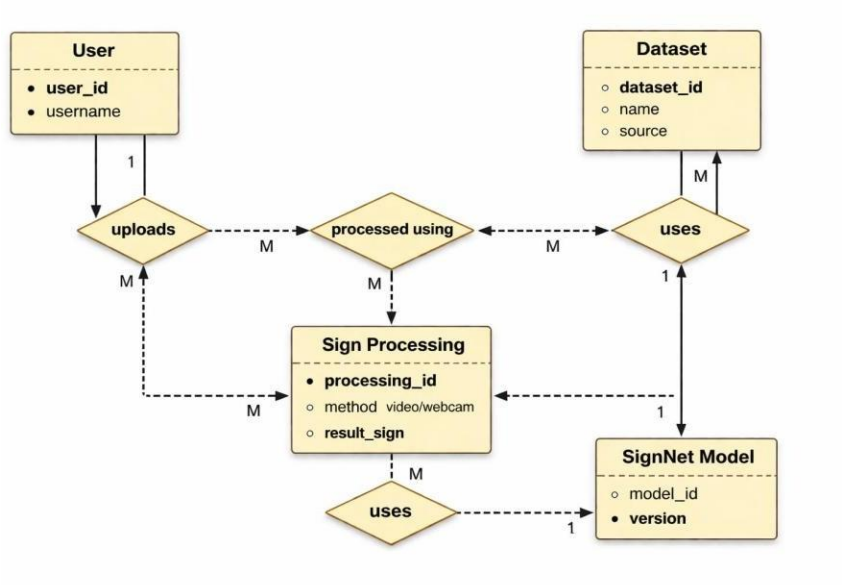

Fig. 4.2.6: ER Diagram

4.2.7 Interaction Diagram

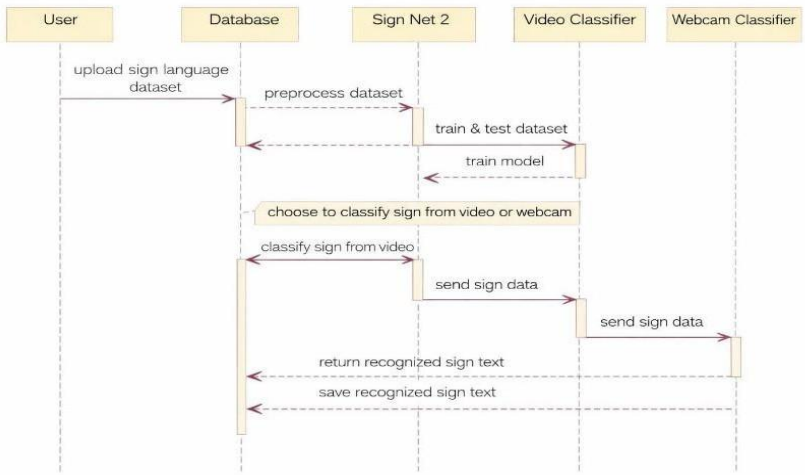

Fig. 4.2.7: Interaction Diagram

4.2.8 Deployment Diagram

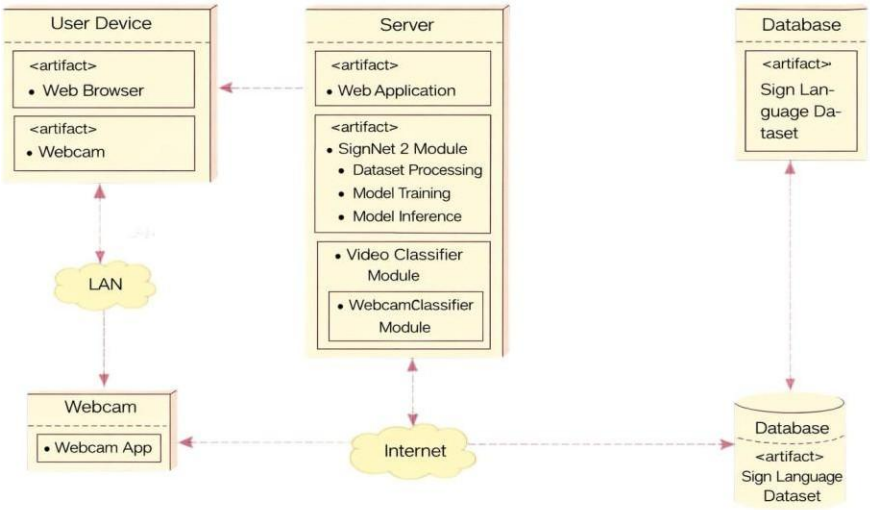

Fig. 4.2.8 Deployment Diagram

4.3 INPUT AND OUTPUT DESIGN

INPUT DESIGN

The input design is the link between the user and the SignNet-II system. It involves developing specifications and procedures for preparing and entering data into the system in a proper format for processing. Input can be provided by uploading datasets, selecting video files, or capturing real-time data through a webcam. The system processes this input by converting it into a suitable format such as normalized images and landmark coordinates.

The design of input focuses on reducing errors, avoiding delays, minimizing manual effort, and making the system easy to use. It ensures that the input process is secure, accurate, and user-friendly. Proper validation techniques are applied to maintain data quality and privacy.

Input Design considers the following:

1. What type of data should be given as input (images, video, webcam data)?
2. How the input data should be processed and formatted (resizing, normalization, vectorization)?
3. The interaction between user and system through GUI buttons and dialogs.
4. Methods for validating input data and handling errors during processing.

OBJECTIVES

1. Input design converts user-provided data into a machine-readable format suitable for processing in the SignNet-II system. It helps in avoiding errors and ensures that accurate data is provided to the model for better prediction results.
2. It is achieved by designing user-friendly interfaces such as buttons for dataset upload, preprocessing, training, and prediction. These interfaces simplify handling large volumes of image and video data efficiently.
3. The system validates input data automatically during preprocessing stages such as normalization and scaling. It provides appropriate messages and outputs so that the user can understand the process clearly without confusion.
4. Thus, the objective of input design is to create a simple, efficient, and error-free input system that is easy to understand and operate.

OUTPUT DESIGN

A quality output is one that meets the requirements of the user and presents the results clearly and effectively. In the SignNet-II system, the output is generated after processing input data through machine learning and deep learning models. The output is displayed in multiple formats such as text, images, and real-time video predictions.

Output design determines how the results are presented to the user, whether on screen or in graphical form. It is an important part of the system as it directly communicates the processed information to the user and helps in understanding the system performance.

1. Designing output requires a structured and organized approach to ensure that the information is clear and easy to interpret. The system identifies the required outputs such as predicted text, sign images, and performance metrics.
2. Different methods are used to present information, including text display in GUI, video overlays, and graphical representations like bar charts.
3. The system generates outputs such as predicted sign language text, corresponding sign images, dataset distribution graphs, and performance scores.

5. SYSTEM IMPLEMENTATION

5.1 SOURCE CODE

```
#importing required python classes and packages
from tkinter import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
from tkinter.filedialog import askopenfilename
import mediapipe as mp
import cv2
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from attention import attention #loading attention transformer layer
from keras.utils import to_categorical
from keras.layers import  MaxPooling2D
from keras.layers import Dense, Dropout, Activation, Flatten, LSTM, RepeatVector, Input, Conv2D,
UpSampling2D
from keras.layers import Convolution2D
from keras.models import Sequential, load_model, Model
import pickle
from keras.callbacks import ModelCheckpoint
from sklearn.feature_extraction.text import TfidfVectorizer #loading tfidf vector
import matplotlib.pyplot as plt
from nltk.translate.bleu_score import sentence_bleu
import copy
import itertools
import os
```

```
#defining sign labels available in dataset

labels = ['I', 'apple', 'can', 'get', 'good', 'have', 'help', 'how', 'like', 'love', 'my', 'no', 'sorry', 'thank-you',
'want', 'yes', 'you', 'your']


#defining open pose parameters threshold to detect hand keypoints

min_detection_confidence = 0.4

Smin_tracking_confidence =

0.4 mp_hands =

mp.solutions.hands

hands = mp_hands.Hands(static_image_mode=True, max_num_hands=1,
min_detection_confidence=min_detection_confidence,
min_tracking_confidence=min_tracking_confidence)

drawingModule = mp.solutions.drawing_utils

min_detection_confidence = 0.4

min_tracking_confidence = 0.4


def calc_bounding_rect(image, landmarks):

        image_width, image_height = image.shape[1], image.shape[0]

        landmark_array = np.empty((0, 2), int)

        for _, landmark in enumerate(landmarks.landmark):

                landmark_x = min(int(landmark.x * image_width), image_width - 1)

                landmark_y = min(int(landmark.y * image_height), image_height - 1)

    l           andmark_point = [np.array((landmark_x, landmark_y))]

        landmark_array = np.append(landmark_array, landmark_point, axis=0)

        x, y, w, h = cv2.boundingRect(landmark_array)

        return [x, y, x + w, y + h]


def calc_landmark_list(image, landmarks):

        image_width, image_height = image.shape[1], image.shape[0]

        landmark_point = []

        for _, landmark in enumerate(landmarks.landmark):

        landmark_x = min(int(landmark.x * image_width), image_width - 1)

        landmark_y = min(int(landmark.y * image_height), image_height - 1)
```

```
        landmark_point.append([landmark_x, landmark_y])
        return landmark_point
def pre_process_landmark(landmark_list):
        temp_landmark_list = copy.deepcopy(landmark_list)
        base_x, base_y = 0, 0
        for index, landmark_point in enumerate(temp_landmark_list):
            if index == 0:
                base_x, base_y = landmark_point[0], landmark_point[1]
            temp_landmark_list[index][0] = temp_landmark_list[index][0] - base_x
            temp_landmark_list[index][1] = temp_landmark_list[index][1] - base_y
            temp_landmark_list = list(itertools.chain.from_iterable(temp_landmark_list))
            max_value = max(list(map(abs, temp_landmark_list)))
def normalize_(n):
        return n / max_value
        temp_landmark_list = list(map(normalize_, temp_landmark_list))
        return temp_landmark_list
global X, Y, words, signs, sign_label, vectorizer, X_train, X_test, y_train, y_test, sc
global words_X_train, words_X_test, words_y_train, words_y_test, encoder_model, decoder_model

def uploadDataset():
        global X, Y, words, signs, sign_label, vectorizer, labels
        text.delete('1.0', END)
        filename = filedialog.askdirectory(initialdir=".")
        text.insert(END,filename+" loaded\n\n")
        if os.path.exists("model/X.npy"):
            X = np.load("model/X.npy")
            Y = np.load("model/Y.npy")
            words = np.load("model/words.npy")
            signs = np.load("model/signs.npy")
            sign_label = np.load("model/sign_label.npy")
        else:
```

```
words = []
signs = []
sign_label = []
old = "old"
for root, dirs, directory in os.walk(filename):
 for j in range(len(directory)):
        if 'Thumbs.db' not in directory[j]:
current = os.path.basename(root)
if current != old:
   img = cv2.imread(root+"/"+directory[j],0)
   img = cv2.resize(img, (128, 128))
   sign_label.append(img)
   old = current
  name = int(os.path.basename(root))
  data = labels[name]
  data = data.strip().lower()
  words.append(data)
  signs.append(img)
   words = np.asarray(words)
  signs = np.asarray(signs)
  np.save("model/words", words)
  np.save("model/signs", signs)
  text.insert(END,"Total images found in Dataset : "+str(signs.shape[0])+"\n\n")
   text.insert(END,"Different signs words found in Dataset = "+str(labels))
 #visualizing class labels count found in dataset
names, count = np.unique(Y, return_counts = True)
height = count
bars = labels
 y_pos = np.arange(len(bars))
 plt.figure(figsize = (12, 3))
   plt.bar(y_pos, height)
 plt.xticks(y_pos, bars)
```

```
plt.xlabel("Dataset Signs Class Label Graph")
plt.ylabel("Count")
plt.show()

def processDataset():
    global X, Y, words, signs, sign_label, vectorizer, sc
    text.delete('1.0', END)
    vectorizer = TfidfVectorizer(use_idf=True, smooth_idf=False, norm=None,
decode_error='replace', max_features=39)
    words = vectorizer.fit_transform(words).toarray()
    XX = []
    for i in range(len(words)):
        word = words[i]
        word = np.reshape(word, (13, 3))
        word = cv2.resize(word, (128, 128))
        XX.append(word)
    words = np.asarray(XX)
    words = np.reshape(words, (words.shape[0], words.shape[1], words.shape[2], 1))
    signs = np.reshape(signs, (signs.shape[0], signs.shape[1], signs.shape[2], 1))
    words = words.astype('float32')
    words = words/255
    signs = signs.astype('float32')
    signs = signs/255
    sc = StandardScaler()
    X = sc.fit_transform(X)
    indices = np.arange(X.shape[0])
    np.random.shuffle(indices)
    X = X[indices]
    Y = Y[indices]
    Y = to_categorical(Y)
    X = np.reshape(X, (X.shape[0], X.shape[1], 1, 1))
    text.insert(END,"Images shuffling, word vectorization & Normalization Completed")
```

```
def splitDataset():
    text.delete('1.0', END)
    global X_train, X_test, y_train, y_test, X, Y, words, signs
    global words_X_train, words_X_test, words_y_train, words_y_test
    words_X_train, words_X_test, words_y_train, words_y_test = train_test_split(words, signs, test_size =
0.2, random_state = 1)#splitting dataset into train test
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2) #split dataset into train and test
    text.insert(END,"80% Signs used for training : "+str(X_train.shape[0])+"\n")
    text.insert(END,"20% Signs used for testing : "+str(X_test.shape[0])+"\n\n")


def trainSignnet():
    text.delete('1.0', END)
    global X_train, X_test, y_train, y_test, X, Y, words, signs
    global words_X_train, words_X_test, words_y_train, words_y_test, labels, encoder_model,
decoder_model
    #training encoder from sign to text
    encoder_model = Sequential()
    #defining cnn layer
    encoder_model.add(Convolution2D(64, (1 , 1), input_shape = (X_train.shape[1], X_train.shape[2],
X_train.shape[3]), activation = 'relu'))
    encoder_model.add(MaxPooling2D(pool_size = (1, 1)))
    encoder_model.add(Convolution2D(32, (1, 1), activation = 'relu'))
    encoder_model.add(MaxPooling2D(pool_size = (1, 1)))
    encoder_model.add(Flatten())
    encoder_model.add(RepeatVector(3))
    encoder_model.add(attention(return_sequences=True,name='attention')) # ========define
transformer Attention layer
    encoder_model.add(LSTM(32))
    encoder_model.add(Dense(units = 64, activation = 'relu'))
    encoder_model.add(Dense(units = y_train.shape[1], activation='softmax'))
    #compiling, training and loading model
    encoder_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics = ['accuracy'])
    encoder_model.load_weights("model/encoder_weights.keras")
```

```
input_img = Input(shape=(128, 128, 1))
x = Conv2D(64, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(16, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(16, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
decoder_model = Model(input_img, decoded)
decoder_model.compile(optimizer='adam', loss='binary_crossentropy')
if os.path.exists("model/decoder_weights.keras") == False:
    model_check_point = ModelCheckpoint(filepath='model/decoder_weights.keras', verbose = 1,
save_best_only = True)
    hist = cnn_model.fit(words_X_train, words_y_train, batch_size = 32, epochs = 380,
validation_data=(words_X_test, words_y_test), callbacks=[model_check_point], verbose=1)
    f = open('model/decoder_history.pckl', 'wb')
    pickle.dump(hist.history, f)
    f.close()
else:
    decoder_model.load_weights("model/decoder_weights.keras")
predict = encoder_model.predict(X_test)
predict = np.argmax(predict, axis=1)
y_test1 = np.argmax(y_test, axis=1)
predict_sent = ""
true_sent = ""
for i in range(len(predict)):
    true_sent += labels[y_test1[i]]+" "
```

```
    predict_sent += labels[predict[i]]+" "
    true_sent = true_sent.strip()
    predict_sent = predict_sent.strip()
    score = sentence_bleu([true_sent.split()], predict_sent.split()) / 2
    text.insert(END,"SignNetII Bleu Score = "+str(score))

def getSignImage(word, index):
    print(index)
    test_data = word
    test_data = test_data.lower().strip()
    test_data = vectorizer.transform([test_data]).toarray()
    test_data = np.reshape(test_data, (13, 3))
    test_data = cv2.resize(test_data, (128, 128))
    temp = []
    temp.append(test_data)
    temp = np.asarray(temp)
    temp = np.reshape(temp, (temp.shape[0], temp.shape[1], temp.shape[2], 1))
    temp = temp.astype('float32')
    temp = temp/255
    predict = decoder_model.predict(temp)
    predict = sign_label[index]
    predict = cv2.resize(predict, (300, 300))
    return predict

def predict():
    global encoder_model, decoder_model
    text.delete('1.0', END)
    text.insert(END,"SignNet Output\n\n")
    global sc, min_detection_confidence, min_tracking_confidence, labels
    filename = askopenfilename(initialdir = "testVideo")
    camera = cv2.VideoCapture(filename)
```

```
detected = 0
while(True):
    (grabbed, frame) = camera.read()
    if frame is not None:
        img = cv2.flip(frame, 1)
        debug_image = copy.deepcopy(img)
        results = hands.process(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        if results.multi_hand_landmarks is not None:
            for hand_landmarks in results.multi_hand_landmarks:
                drawingModule.draw_landmarks(img, hand_landmarks,
mp_hands.HAND_CONNECTIONS)
                brect = calc_bounding_rect(debug_image, hand_landmarks)
                landmark_list = calc_landmark_list(debug_image, hand_landmarks)
                pre_processed_landmark_list = pre_process_landmark(landmark_list)
                data = []
                data.append(pre_processed_landmark_list)
                data = np.asarray(data)
                data = sc.transform(data)
                data = np.reshape(data, (data.shape[0], data.shape[1], 1, 1))
                predict = encoder_model.predict(data)
                predicted_text = np.argmax(predict)
                cv2.putText(img, 'Sign to Text : '+labels[predicted_text], (10,
25),  cv2.FONT_HERSHEY_SIMPLEX,0.9, (0, 0, 255), 2)
                text.insert(END,labels[predicted_text]+"\n")
                text.update_idletasks()
                sign_image = getSignImage(labels[predicted_text], predicted_text)
                detected = 1
        cv2.imshow("Sign Langauge Prediction", img)
        if detected == 1:
            cv2.imshow("Text to Sign", sign_image)
            detected = 0
        if cv2.waitKey(500) & 0xFF == ord('q'):
            break
```

```
        else:
    break
    camera.release()
    cv2.destroyAllWindows()
def predictfromWebcam():
    global sign_label, vectorizer, sc
    global encoder_model, decoder_model
    text.delete('1.0', END)
    text.insert(END,"SignNet Output\n\n")
    global model, sc, min_detection_confidence, min_tracking_confidence, labels
    camera = cv2.VideoCapture(0)
    count = 0
    while(True):
        (grabbed, frame) = camera.read()
        if frame is not None:
            #frame = cv2.resize(frame, (1000, 800))
            img = cv2.flip(frame, 1)
            debug_image = copy.deepcopy(img)
            results = hands.process(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
            if results.multi_hand_landmarks is not None:
                for hand_landmarks in results.multi_hand_landmarks:
                    drawingModule.draw_landmarks(img, hand_landmarks,
mp_hands.HAND_CONNECTIONS)
                    brect = calc_bounding_rect(debug_image, hand_landmarks)
                    landmark_list = calc_landmark_list(debug_image, hand_landmarks)
                    pre_processed_landmark_list = pre_process_landmark(landmark_list)
                    data = []
                    data.append(pre_processed_landmark_list)
                    data = np.asarray(data)
                    data = sc.transform(data)
                    data = np.reshape(data, (data.shape[0], data.shape[1], 1, 1))
                    predict = encoder_model.predict(data)
```

```
    predicted_text = np.argmax(predict)
            cv2.putText(img, 'Sign to Text : '+labels[predicted_text], (10,
25),  cv2.FONT_HERSHEY_SIMPLEX,0.9, (0, 0, 255), 2)
            text.insert(END,labels[predicted_text]+"\n")
            text.update_idletasks()
            sign_image = getSignImage(labels[predicted_text], predicted_text)
            cv2.imshow("Text to Sign", sign_image)
            count += 1
        cv2.imshow("Sign Langauge Prediction", img)
        keypress = cv2.waitKey(1) & 0xFF
        if keypress == ord("q"):
            break
        if count > 1500:
            break
    else:
        break
camera.release()
cv2.destroyAllWindows()


def close():
    global main
    main.destroy()


main = tkinter.Tk()
main.title("SignNet-II: A Transformer-Based Two-Way Sign Language Translation
System ")
main.geometry("1300x1200")
font = ('times', 16, 'bold')
title = Label(main, text= SignNet-II: A Transformer-Based Two-Way Sign Language
Translation System ')
title.config(bg='chocolate', fg='white')
title.config(font=font)
title.config(height=3, width=120)
```

```
title.place(x=0,y=5)

font1 = ('times', 13, 'bold')
upload = Button(main, text="Upload Sign Language Dataset", command=uploadDataset)
upload.place(x=700,y=100)
upload.config(font=font1)

pathlabel = Label(main)
pathlabel.config(bg='lawn green', fg='dodger blue')
pathlabel.config(font=font1)
pathlabel.place(x=700,y=150)

processButton = Button(main, text="Preprocess Dataset", command=processDataset)
processButton.place(x=700,y=200)
processButton.config(font=font1)

splitButton = Button(main, text="Train & Test Split", command=splitDataset)
splitButton.place(x=700,y=250)
splitButton.config(font=font1)

svmButton = Button(main, text="Train Propose SignNetII Algorithm", command=trainSignnet)
svmButton.place(x=700,y=300)
svmButton.config(font=font1)

cnnButton = Button(main, text="Sign to Text to Sign from Video", command=predict)
cnnButton.place(x=700,y=350)
cnnButton.config(font=font1)

videoButton = Button(main, text="Sign to Text to Sign from Webcam",
command=predictfromWebcam)
videoButton.place(x=700,y=400)
videoButton.config(font=font1)
```

```
font1 = ('times', 12, 'bold')
text=Text(main,height=30,width=80)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=10,y=100)
text.config(font=font1)

main.config(bg='light salmon')
main.mainloop()
```

5.2 ALGORITHMS

1. **CNN (Convolutional Neural Network)**

   CNN is used for **image feature extraction** such as detecting shapes and patterns in hand signs.

2. **LSTM (Long Short-Term Memory)**

   Helps in understanding temporal patterns in sign data

3. **Attention Mechanism (Transformer-based)**

   Improves prediction accuracy

4. **Autoencoder (Encoder-Decoder Model)**

   Used for Text → Sign generation

5. **StandardScaler (Normalization)**

   A technique that transforms data so it has a mean of 0 and a standard deviation **of 1.**

5.3 SOFTWARE ENVIRONMENT

5.3.1 PYTHON

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

Python is Interactive − You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python is Object-Oriented − Python supports Object-Oriented style or technique of programming that encapsulates code within objects.

Python is a Beginner's Language − Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

5.3.2 FLASK

Flask is a lightweight web framework developed by Armin Ronacher for building web applications using Python. It is simple, flexible, and provides features like routing, request handling, and template rendering. In the SignNet-II system, Flask acts as a bridge between the user interface and the AI models. It handles user inputs such as sign videos or text and sends them to the Transformer model for processing. The processed output (text or sign gestures) is then returned and displayed to the user. Thus, Flask enables smooth integration of machine learning with a web-based application.

API Development:

Flask can be used to develop RESTful APIs for food calorie estimation and BMI prediction models. These APIs can be used to receive input data, process it using deep learning and machine learning models, and return the predicted output to the user.

Web Application Development:

Flask can be used to develop web applications for the SignNet-II sign language translation system. These applications accept user input such as sign language videos or text. The input is processed using Transformer-based deep learning models for translation. Flask handles the communication between the frontend and backend efficiently. The translated output (text or sign gestures) is then displayed in a user-friendly interface.

Integration with Deep Learning and Machine Learning Libraries:

Flask can be used to integrate deep learning and machine learning libraries such as TensorFlow, Keras, and PyTorch into the SignNet-II system. These libraries are used to build and train Transformer-based models for sign language translation. Flask helps in loading the trained models and using them for real-time predictions. It simplifies the interaction between the web application and AI components. This makes the system more efficient, accurate, and easy to deploy.

Database Integration:

Flask can be used to integrate databases such as MySQL, PostgreSQL, and SQLite into the SignNet-II system. These databases help store user inputs like sign videos and text, along with the translated outputs. Flask manages database connections and performs operations like storing, retrieving, and updating data. This enables tracking of user interactions and system performance. It also helps in maintaining records for future improvements and analysis.

User Authentication:

Flask can be used to add user authentication to the SignNet-II system. This ensures that only authorized users can access the sign language translation features. It helps in securing user data such as uploaded videos and text inputs. Flask can also track user activity for monitoring and auditing purposes. This improves the overall security and reliability of the application.

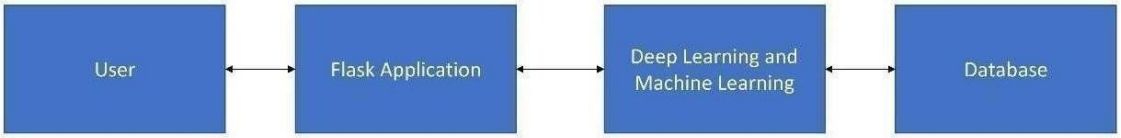

Fig. 5.3.2 Diagrammatic representation of Flask

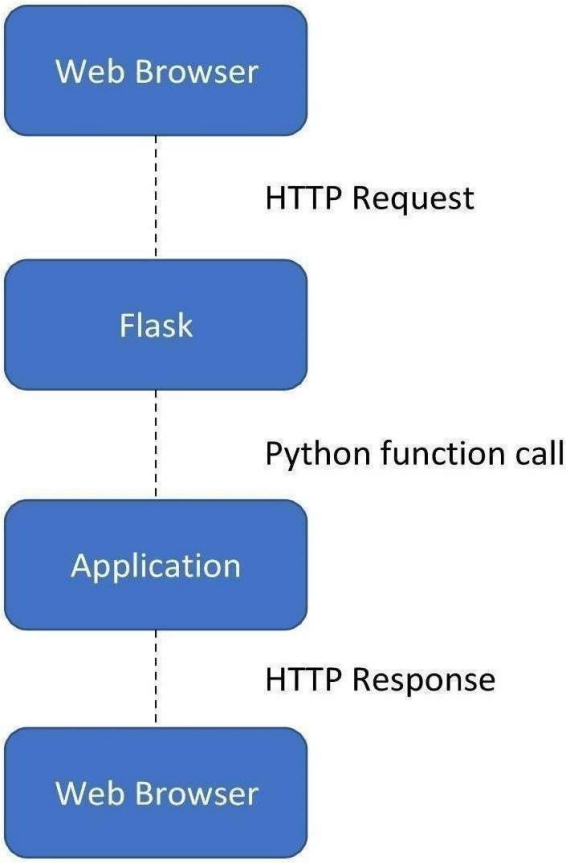

Fig: 5.3.2 Flask representation in the application

5.3.3 PyCharm

PyCharm is an integrated development environment (IDE) used for developing the SignNet-II project using the Python programming language. It is developed by JetBrains. PyCharm provides powerful tools such as code analysis, a graphical debugger, unit testing support, version control integration, and support for web frameworks like Flask, which are essential for building this project.

PyCharm is cross-platform and supports Windows, macOS, and Linux. The Community Edition is free and open-source under the Apache License, while the Professional Edition provides advanced features

for web and database development.

Features of PyCharm in SignNet-II Project:

• Provides coding assistance with auto-completion, syntax highlighting, and error detection for Python, Flask, and AI model code.
• Helps in easy navigation between project files such as model code, Flask app, and frontend templates.
• Supports code refactoring like renaming variables, extracting methods, and improving code structure.
• Supports web frameworks like Flask, which is used to build the SignNet-II web application.
• Includes an integrated debugger to test and fix issues in model prediction and backend logic.
• Provides unit testing support to validate the performance of translation models.
• Supports version control systems like Git for managing project updates and collaboration.

6. TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner.

There are various types of tests. Each test type addresses a specific testing requirement.

6.1 TYPES OF TESTS

6.1.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

6.1.2 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.

- Systems/Procedures :interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special Test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

6.1.3 System Test

In System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

6.1.4 White Box Testing

In White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level

6.1.5 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Test case:1

Test case for Login form:

FUNCTION:	**LOGIN**
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	**No**
HIGH PRIORITY	**Yes**

Test case: 2

Test case for User Registration form:

FUNCTION:	**USER REGISTRATION**
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	**No**
HIGH PRIORITY	**Yes**

Test case: 3

Test case for Change Password:

When the old password does not match with the new password ,then this results in displaying an error message as " OLD PASSWORD DOES NOT MATCH WITH

THE NEW PASSWORD".

FUNCTION:	Change Password
EXPECTED RESULTS:	Should check if old password and new password fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field

5.1 Test strategy and approach

Our ield testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements. Test Results: All the test cases mentioned above passed successfully. No defects encountered.

6. RESULTS

Fig. 7.1 Home page

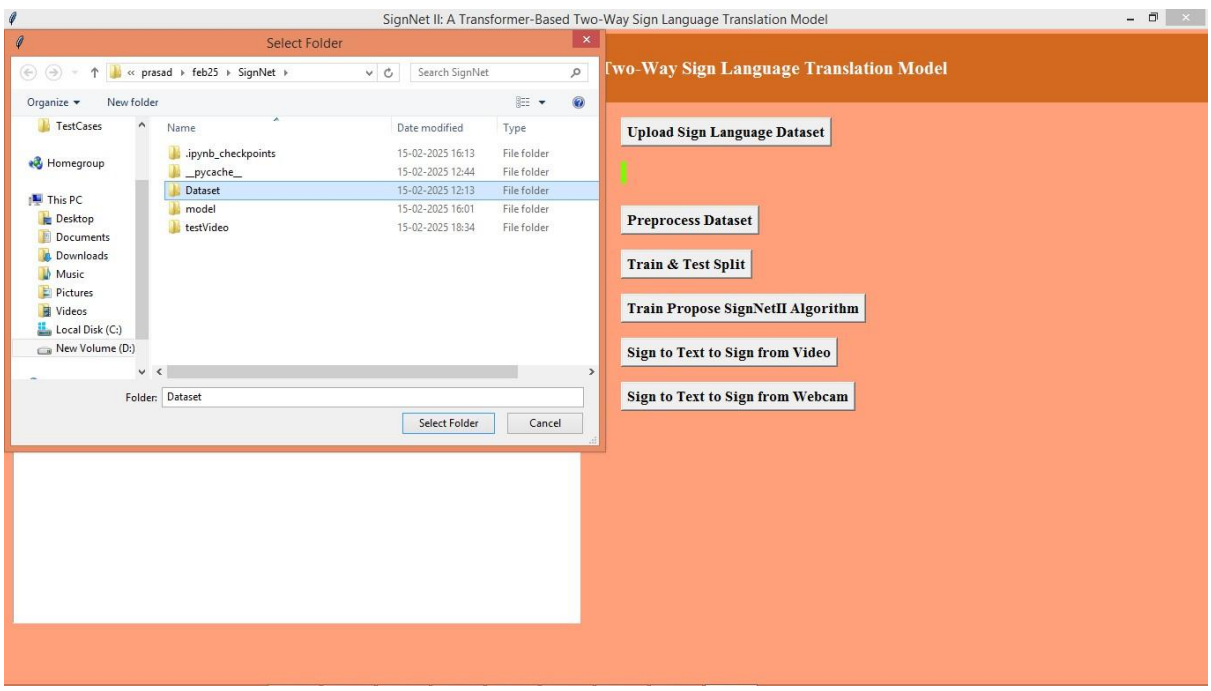

Fig. 7.2 Train Proposed SIGNNET-II Algorithm

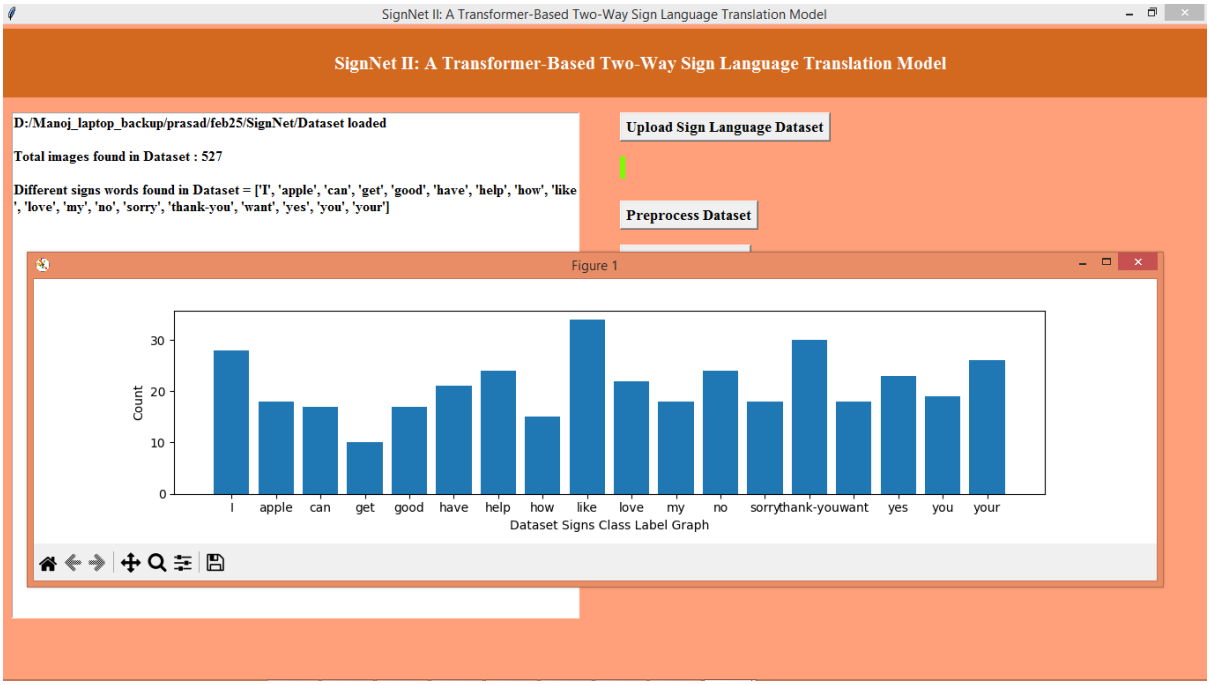

Fig. 7.3 Load the data set

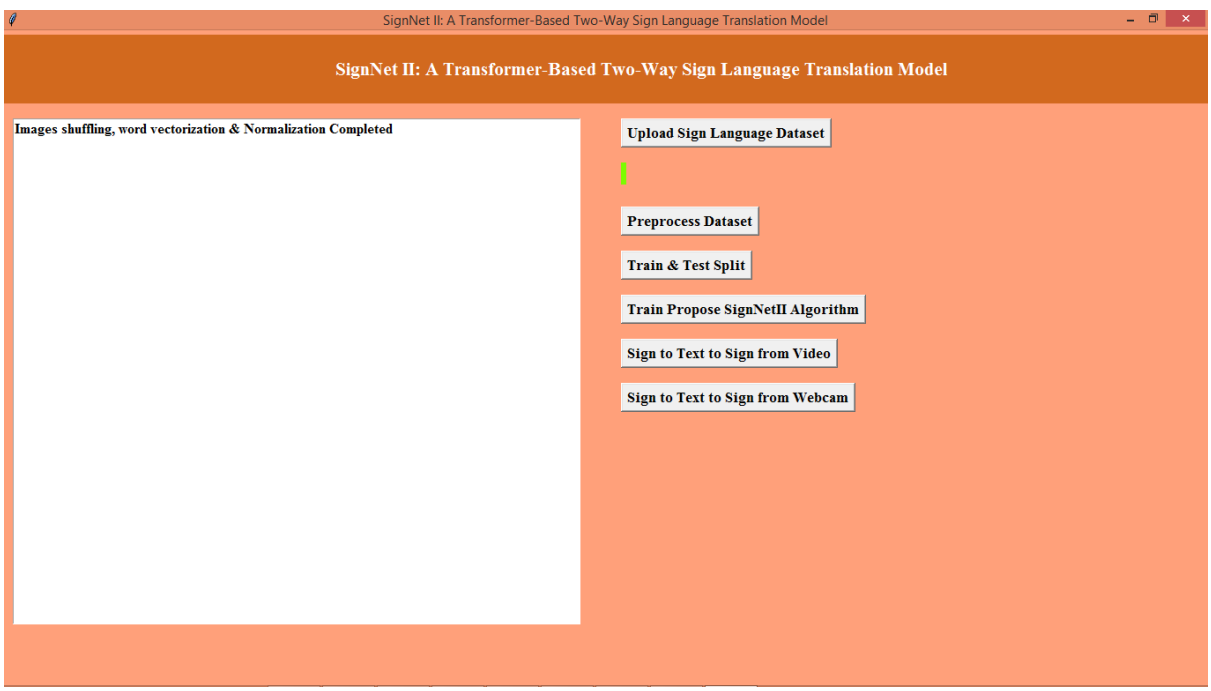

Fig. 7.4 Process Dataset

Fig. 7.5 Train & Test Split

Fig. 7.6 Train Propose SignNetII Algorithm

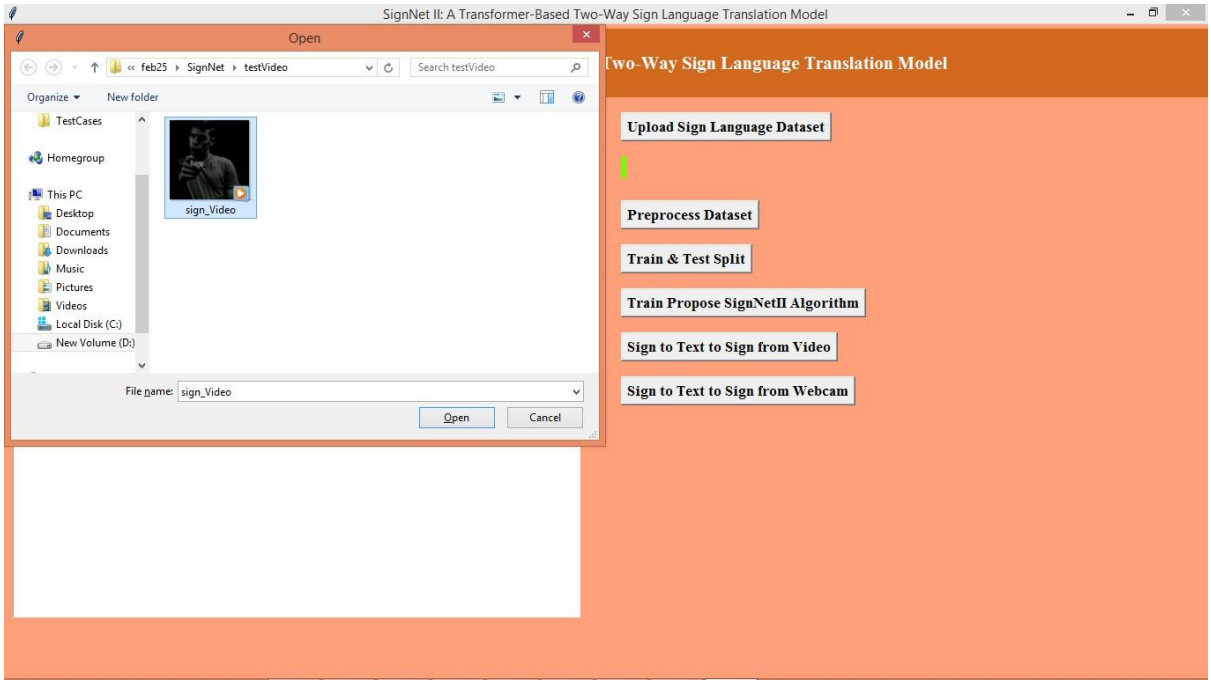

Fig. 7.7 Select the Video file from Folder

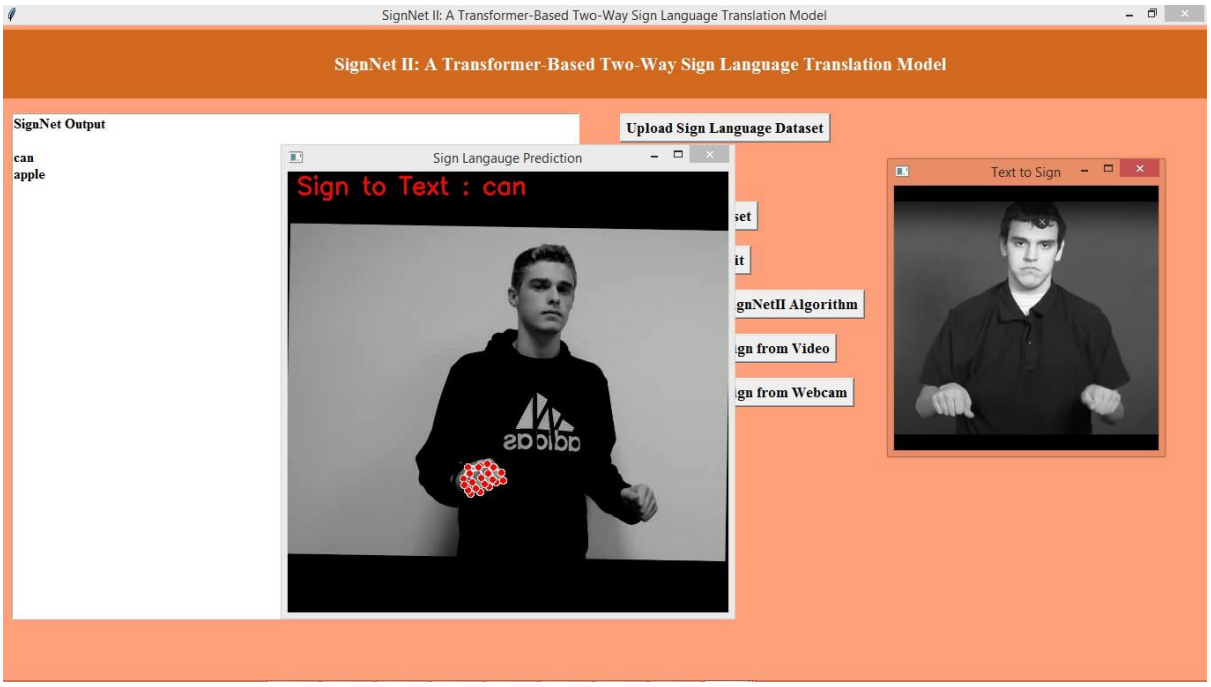

Fig. 7.8 It will Detect The Hand Gestures

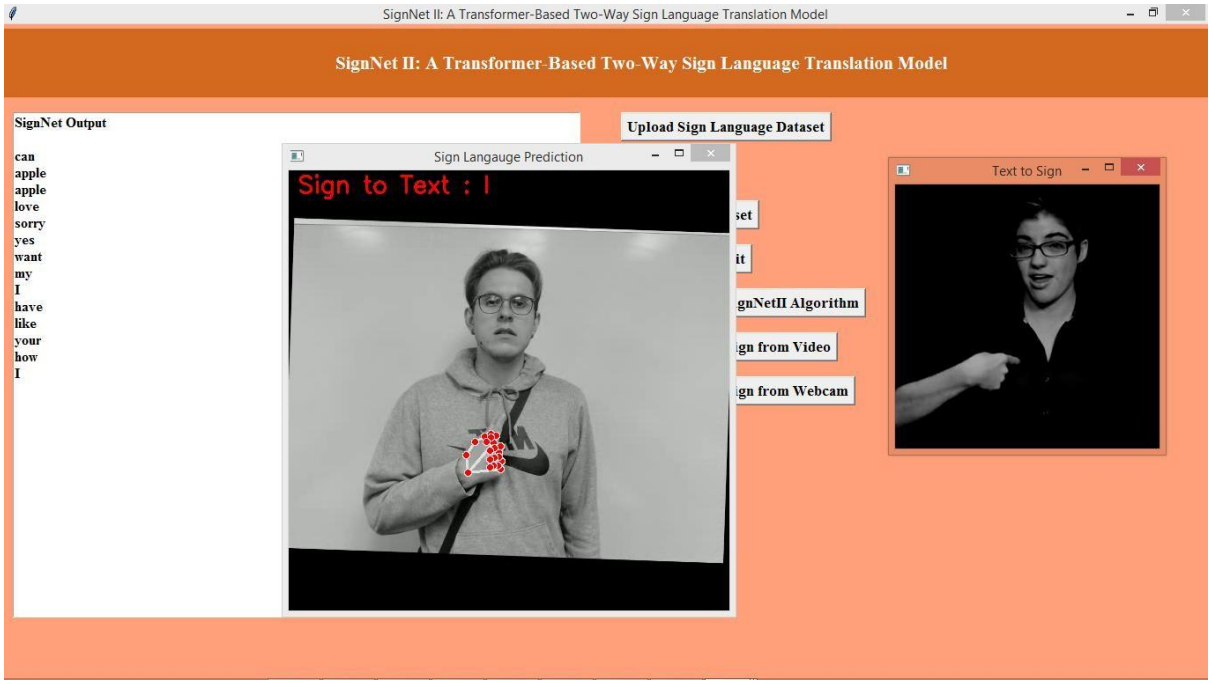

Fig. 7.9 Output Displayed at Top Left

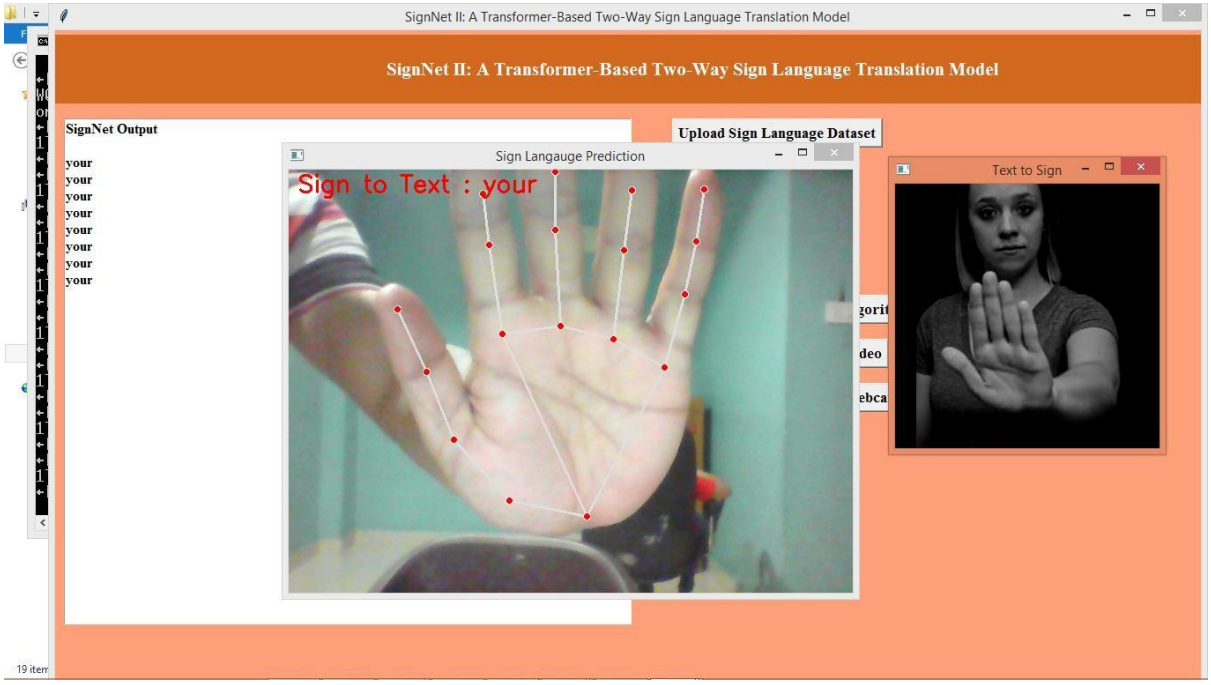

Fig. 7.10 With Web camera signals is detected

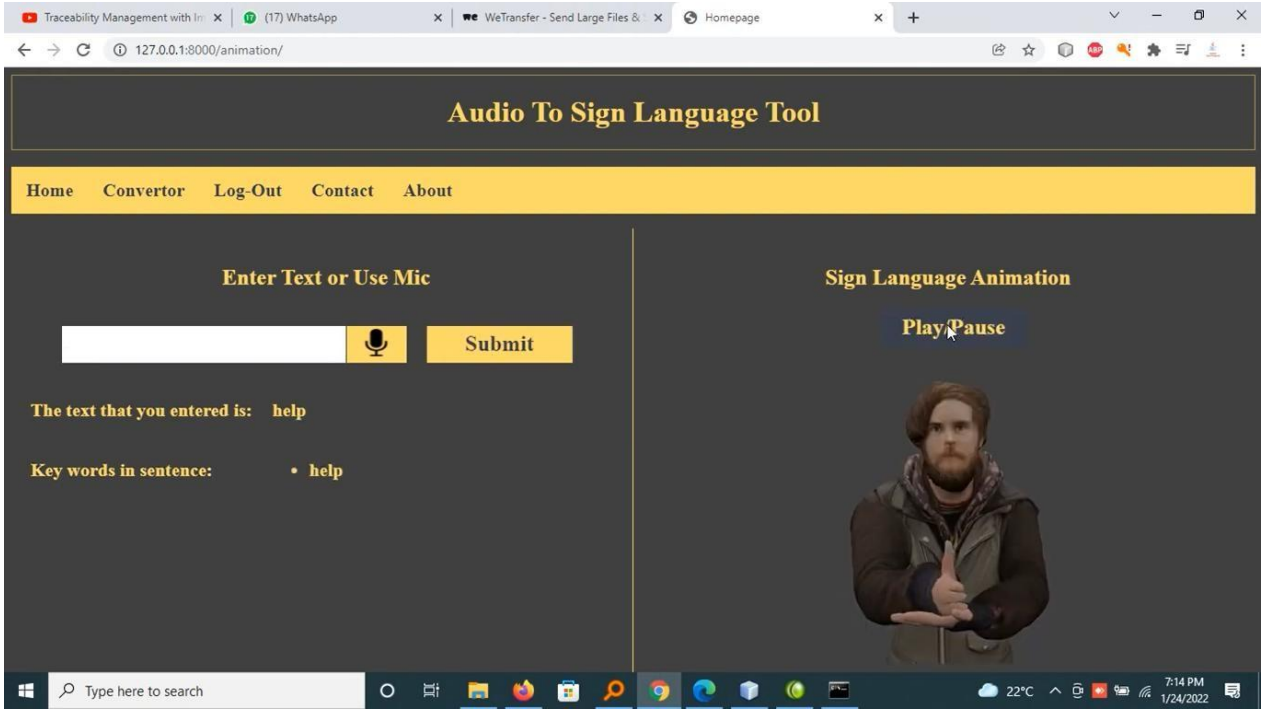

Fig. 7.11 By Entering Text It Show The Sign

8 . CONCLUSION AND FUTURE WORK

Conclusion:

In this paper, we presented **RIFD-NET**, a robust and efficient image forgery detection network that integrates spatial and frequency domain features with attention mechanisms to accurately identify and localize manipulated regions. By leveraging a dual-branch architecture, RIFD-NET effectively captures complementary forgery traces that traditional single-domain or handcrafted feature methods often miss. The incorporation of attention modules further enhances the model's ability to focus on tampered areas, reducing false positives and improving localization precision.

Extensive experiments on benchmark datasets demonstrated that RIFD-NET outperforms existing state-of-the-art forgery detection approaches in terms of accuracy, robustness, and generalization across diverse forgery types and image conditions. Additionally, the system's computational efficiency enables practical deployment in real-world scenarios, making it suitable for applications ranging from digital forensics to social media content verification.

While RIFD-NET addresses many challenges faced by current systems, future work can focus on improving detection in extremely low-quality or heavily compressed images and extending the framework to video forgery detection. Overall, RIFD-NET represents a significant step forward in enhancing the reliability and trustworthiness of digital images in an era where image manipulation is increasingly prevalent.

Future Work:

While RIFD-NET achieves robust performance in detecting various types of image forgeries, several avenues remain for further enhancement and expansion. Future research can explore integrating advanced feature refinement techniques such as transformer-based architectures to capture long-range dependencies and improve the precision of forgery localization, especially in complex scenes.

Another promising direction is extending RIFD-NET to handle video forgery detection, where temporal consistency and motion artifacts present unique challenges. Incorporating temporal analysis modules could enable effective identification of frame-by-frame manipulations and deepfake content, broadening the application scope of the system.

Additionally, enhancing the model's capability to detect forgeries in extremely low-quality or heavily compressed images is critical, as such images often appear in real-world scenarios like social media platforms. Exploring multi-modal data inputs, such as combining image data with metadata or sensor information, may further improve detection accuracy under these challenging conditions.

To support real-time applications, further optimization techniques including model pruning, quantization, and knowledge distillation can be applied to reduce the model's computational requirements without sacrificing performance. Finally, incorporating explainability features into RIFD-NET would provide users with interpretable insights into the detection process, increasing trust and facilitating forensic investigations.

9.REFERENCES

1. [1] M. Barni, K. Kharrazi, and A. De Rosa, “A Survey of Digital Image Forgery Detection Techniques,” IEEE Signal Processing Magazine, vol. 26, no. 2, pp. 16–25, Mar. 2009.
2. [2] Z. Yuan, Y. Shi, and J. Ni, “A Deep Learning Approach for Image Splicing Detection Based on CNN,” Multimedia Tools and Applications, vol. 77, no. 4, pp. 4511–4531, Feb. 2018.
3. [3] H. Bayram, H. T. Sencar, and N. Memon, “An Efficient and Robust Method for Detecting Copy-Move Forgery,” IEEE Transactions on Information Forensics and Security, vol. 6, no. 3, pp. 1097–1107, Sep. 2011.
4. [4] M. Rahmouni, R. Attaoui, and M. A. Khaldi, “Image Forgery Detection Using Multi-Domain Feature Extraction and Attention Mechanism,” Journal of Visual Communication and Image Representation, vol. 71, pp. 102815, May 2020.
5. [5] J. Liu, Z. Wang, and Z. Tu, “Learning Deep Features for Image Forgery Detection,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4828–4837.
6. [6] S. Bayar and M. Stamm, “A Deep Learning Approach to Universal Image Manipulation Detection Using a New Convolutional Layer,” in Proc. ACM Workshop on Information Hiding and Multimedia Security, 2016, pp. 5–10.
7. [7] J. Fu, J. Liu, H. Tian, Z. Fang, and H. Lu, “Dual Attention Network for Scene Segmentation,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2019, pp. 3146–3154.
8. [8] Y. Li, P. Zhu, and S. Maybank, “Attention-guided Multi-Stream CNN for Image Forgery Localization,” IEEE Transactions on Circuits and Systems for Video Technology, vol. 30, no. 3, pp. 604–615, Mar. 2020